Diseño e Implementación de la API REST de la Plataforma Web TasteBox (Caso 6 – Proyecto Bases de Datos, Segundo Corte)

Camilo Arbey Cuadros Mendez, Juan Diego Sierra
Ingeniería Multimedia
Universidad de San Buenaventura
Bogotá, Colombia

***Resumen*—Este documento extiende el análisis de requerimientos de TasteBox (corte 1) con el diseño de la capa de datos y de la API. Se formaliza el modelo entidad–relación y se argumenta el proceso de normalización hasta la forma normal de Boyce–Codd; se mapean las entidades a recursos REST y se justifican las decisiones de anidamiento, referencias y endpoints compuestos; y se describe la arquitectura por capas (rutas, controladores, servicios y repositorios) conectada a MySQL. La implementación en Node.js y Express cubre los diecinueve requerimientos funcionales, se documenta con OpenAPI y Swagger UI y se verifica con una colección de pruebas con casos de éxito y de error por endpoint.**

***Index Terms*—TasteBox, API REST, modelo entidad–relación, normalización, MySQL, Express, OpenAPI.**

XIII. Modelo de datos y diseño de la base de datos

XIII-A. De los recursos preliminares al modelo entidad–relación

En el corte 1 se identificaron, a nivel conceptual, los recursos Usuario, Receta, Colección, Preparación, Reseña, Suscripción y Caja semanal, y se dejó abierta la pregunta de si los hábitos de cocina constituían un recurso almacenado. La formalización de esos recursos en un modelo entidad–relación obliga a resolver tres decisiones que el análisis conceptual no necesitaba tomar.

La primera es que los *hábitos* no se modelan como entidad. Los indicadores exigidos por HU-09 (recetas por mes, distribución por tipo de cocina y racha) son agregados de las preparaciones; almacenarlos duplicaría información derivable y obligaría a mantenerla sincronizada. Por eso se calculan en consulta y se exponen como un recurso de solo lectura (Sección XIV).

La segunda es que aparecen entidades que en el corte 1 estaban implícitas como atributos: *Plan* (el caso distingue un plan gratuito con límite de recetas de uno Premium), *Tipo de cocina* y *Etiqueta* (HU-07 permite etiquetar reseñas). Se modelan como entidades propias porque son dominios que el administrador debe poder mantener y porque su repetición como texto en otras tablas produciría anomalías de actualización (Sección XIII-C).

La tercera es que las relaciones de muchos a muchos se resuelven con tablas asociativas: `colecciones` (Usuario–Receta), `resena_etiquetas` (Reseña–Etiqueta) y `caja_recetas` (Caja–Receta). Con ello el modelo queda con doce relaciones, que se muestran en la Fig. 1.

Tabla I
Cardinalidades y política de borrado referencial

Vínculo (padre → hijo)	Card.	Al borrar el padre
Usuario → Suscripción	1:N	CASCADE
Plan → Suscripción	1:N	RESTRICT
Tipo de cocina → Receta	1:N	RESTRICT
Usuario → Colección	1:N	CASCADE
Receta → Colección	1:N	CASCADE
Usuario → Preparación	1:N	CASCADE
Receta → Preparación	1:N	CASCADE
Usuario → Reseña	1:N	CASCADE
Receta → Reseña	1:N	CASCADE
Reseña → Reseña–Etiqueta	1:N	CASCADE
Etiqueta → Reseña–Etiqueta	1:N	CASCADE
Usuario → Caja semanal	1:N	CASCADE
Caja → Caja–Receta	1:N	CASCADE
Receta → Caja–Receta	1:N	CASCADE

XIII-B. Cardinalidades y reglas de integridad

La Tabla I resume las cardinalidades y la regla de borrado referencial elegida para cada vínculo. La política de borrado no es uniforme porque responde a la naturaleza de cada dependencia: los datos personales y de actividad de un usuario dependen de su cuenta y se eliminan con ella (`CASCADE`), mientras que los catálogos referenciados (planes y tipos de cocina) se protegen con `RESTRICT`, de modo que no pueda borrarse un plan o una cocina en uso. El efecto práctico se verifica en la API: eliminar un tipo de cocina con recetas asociadas responde 409.

Las cardinalidades N:M resultantes son Usuario–Receta (por guardado, por preparación y por reseña), Reseña–Etiqueta y Caja–Receta. Además de las claves, el esquema codifica reglas del caso como restricciones declarativas: el correo es único (HU-01); un usuario solo puede reseñar una vez cada receta, mediante `UNIQUE(id_usuario,id_receta)` (HU-07); una caja es única por usuario y semana; y `CHECK` limita la calificación a 1–5 y el tiempo de preparación a valores positivos.

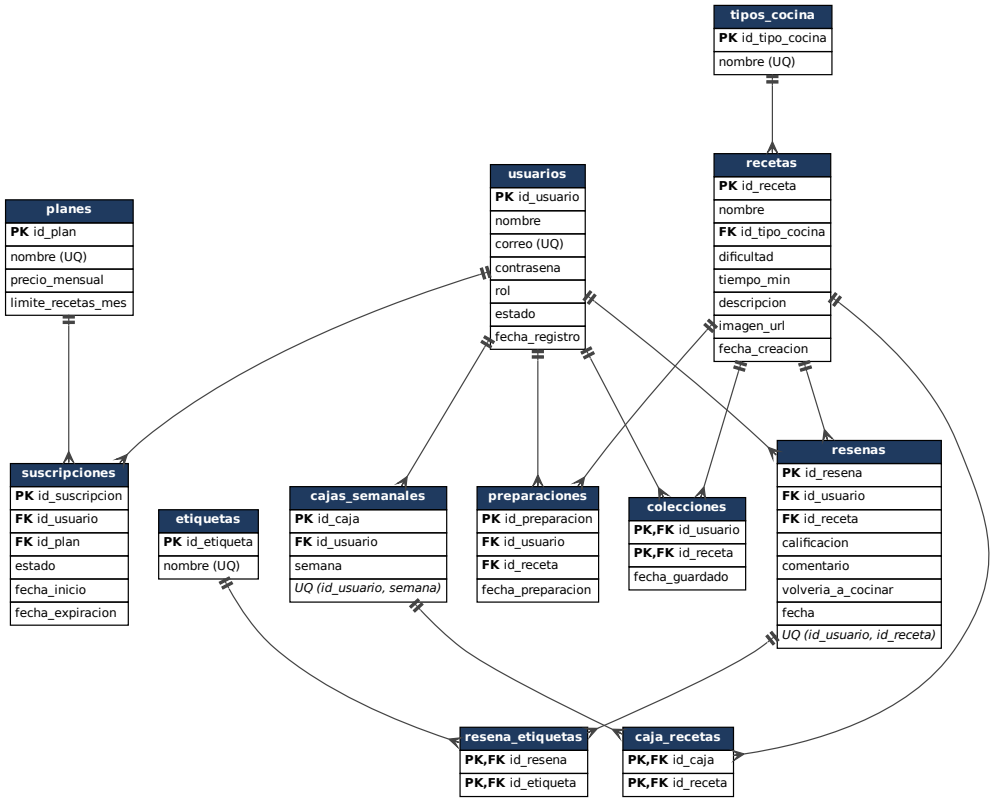


Figura 1. Diagrama entidad–relación de TasteBox. PK: clave primaria; FK: clave foránea; UQ: restricción de unicidad. En cada vínculo, el extremo con doble barra indica "uno" y la pata de gallo indica "muchos".

Dos decisiones merecen justificación explícita. Primero, `preparaciones` usa una clave sustituta y *no* una clave compuesta, porque un usuario puede preparar la misma receta muchas veces y cada ocurrencia es un hecho distinto (alimenta la racha y los conteos mensuales); en cambio `colecciones` sí usa clave compuesta porque guardar dos veces la misma receta no tiene sentido. Segundo, las columnas de dominio cerrado y pequeño (`rol`, `estado`, `dificultad`) se definen como `ENUM` en lugar de tablas de consulta: su conjunto de valores forma parte de la lógica de la aplicación y no lo administra el usuario, a diferencia de los tipos de cocina y las etiquetas. El costo asumido es que agregar un valor exige modificar el esquema.

XIII-C. Proceso de normalización

La normalización se aplicó partiendo de una relación universal que reúne los atributos de todas las historias de usuario. Se muestra de forma abreviada:

```
R0(id_usuario, nombre, correo, contrasena, rol,
   plan_nombre, plan_precio, plan_limite,
fecha_inicio, fecha_exp,
   id_receta, nombre_receta, tipo_cocina,
dificultad, tiempo,
   calificacion, comentario, {etiquetas},
fecha_guardado,
   {fechas_preparacion}, semana_caja,
{recetas_caja})
```

Esta relación presenta las tres anomalías clásicas. Hay anomalía de inserción, porque no puede registrarse una receta del catálogo mientras ningún usuario la haya reseñado o guardado (HU-15 exige lo contrario). Hay anomalía de actualización, porque cambiar el precio del plan Premium obligaría a modificar todas las filas de todos los suscriptores, y escribir "Italiana" de dos formas distintas rompería los filtros de HU-03. Y hay anomalía de borrado, porque eliminar la última reseña de una receta eliminaría también los datos de la receta. Los pasos siguientes eliminan cada causa.

Primera forma normal (1FN). R0 contiene atributos multivaluados (las etiquetas de una reseña, las fechas de preparación y las recetas de una caja). Cada uno se extrae a una relación donde cada celda guarda un solo valor: `resena_etiquetas`, `preparaciones` y `caja_recetas`. Con ello todos los atributos son atómicos y cada relación tiene clave primaria.

Tabla II
CLAVES CANDIDATAS Y DEPENDENCIAS FUNCIONALES

Relación	Claves candidatas	Dependencias no triviales
usuarios	id_usuario; correo	id_usuario → resto; correo → resto
planes	id_plan; nombre	id_plan ↔ nombre; → precio, límite
suscripciones	id_suscripcion	→ usuario, plan, estado, fechas
tipos_cocina	id_tipo_cocina; nombre	id_tipo_cocina ↔ nombre
recetas	id_receta	→ nombre, cocina, dificultad, tiempo, …
colecciones	(id_usuario, id_receta)	→ fecha_guardado
preparaciones	id_preparacion	→ usuario, receta, fecha
resenas	id_resena; (id_usuario, id_receta)	→ calificación, comentario, volvería, fecha
resena_etiquetas	(id_resena, id_etiqueta)	ninguna
cajas_semanales	id_caja; (id_usuario, semana)	→ usuario, semana
caja_recetas	(id_caja, id_receta)	ninguna

Segunda forma normal (2FN). Se detectan dependencias parciales sobre claves compuestas. En la relación Reseña, con clave candidata (id_usuario, id_receta), el nombre y la dificultad de la receta dependen solo de id_receta, y el correo solo de id_usuario. Se separan las entidades usuarios y recetas, y las relaciones resenas, colecciones y preparaciones conservan únicamente las claves foráneas y los atributos propios del hecho (calificación, comentario, fechas). Las relaciones asociativas colecciones, resena_etiquetas y caja_recetas cumplen 2FN porque su único atributo no clave, cuando existe, depende de la clave completa.

Tercera forma normal (3FN). Quedan dependencias transitivas. En suscripciones, id_plan determina nombre, precio y límite, y id_suscripcion solo determina esos valores a través de id_plan; se extrae planes. Del mismo modo, en recetas, el nombre de la cocina depende de id_tipo_cocina y no de la receta; se extrae tipos_cocina. Esta decisión resuelve la anomalía de actualización descrita arriba: el precio de un plan o el nombre de una cocina se almacena una sola vez.

Forma normal de Boyce–Codd (BCNF). Se verificó que todo determinante sea superclave. La Tabla II lista las claves candidatas y las dependencias funcionales no triviales. En usuarios existen dos determinantes, id_usuario y correo, y ambos son claves candidatas; lo mismo ocurre en resenas con id_resena y el par (id_usuario, id_receta). Como no hay un determinante que no sea clave, el esquema alcanza BCNF, que es más estricta que la 3FN pedida como mínimo.

XIII-D. Desnormalización y limitaciones asumidas

No se aplicó ninguna desnormalización en las tablas. La única tentación evidente era guardar en recetas la calificación promedio y el número de reseñas para acelerar el catálogo. Se descartó porque duplicaría un dato derivable, exigiría actualizarlo en cada alta, edición o baja de reseña (con riesgo de inconsistencia si alguna ruta lo olvida) y no es necesario para cumplir RNF-01: el promedio se calcula con AVG() y COUNT() en la misma consulta del catálogo, con tiempos de respuesta de pocos milisegundos en las pruebas realizadas. Si el volumen creciera de forma que ese cálculo incumpliera el umbral de 3 s, la columna redundante sería la primera optimización a considerar, acompañada de un disparador o de una actualización transaccional.

Se asumen también tres limitaciones deliberadas de este corte. (i) *La contraseña se almacena en texto plano*, como exige la definición del corte; esto incumple RNF-03 y se acepta únicamente en este entregable. La corrección prevista consiste en reemplazar la comparación directa por una función de derivación adaptativa como Argon2id o bcrypt con sal [1], [2], sin cambiar el esquema salvo la longitud de la columna. (ii) La regla "una sola suscripción activa por usuario" no se impone en la base de datos, porque MySQL no ofrece índices únicos parciales; se garantiza en la capa de servicios, que cancela la suscripción vigente y crea la nueva dentro de una transacción, conservando el historial. (iii) El borrado de una receta elimina en cascada sus preparaciones, lo que altera los hábitos históricos; en producción sería preferible un borrado lógico.

XIV. IDENTIFICACIÓN Y DISEÑO DE RECURSOS REST

XIV-A. Criterios de diseño

El diseño sigue el estilo REST propuesto por Fielding [3] y la semántica de métodos y códigos de estado de HTTP [4]. Se adoptaron cuatro criterios: (1) los recursos se nombran con sustantivos en plural y las acciones se expresan con el verbo HTTP, no con la URI; (2) GET no modifica estado; PUT reemplaza o actualiza de forma idempotente; POST crea; DELETE elimina; (3) cada resultado se comunica con un código de estado significativo (201 al crear, 204 al eliminar, 400, 401, 403, 404 y 409 para los distintos errores); y (4) la URI refleja la propiedad de los datos, de modo que una colección que pertenece a un usuario cuelga de ese usuario.

XIV-B. Mapeo de entidades a recursos

No existe una correspondencia uno a uno entre tablas y recursos, y esa asimetría es una decisión de diseño. La Tabla III muestra cómo se resolvió cada entidad y por qué.

XIV-C. Resolución de las relaciones entre recursos

Se emplearon tres mecanismos, escogidos según el tipo de relación. El **anidamiento** se usa cuando el hijo no existe sin el padre o le pertenece (reseñas de una receta; colección, preparaciones, suscripción y caja de un usuario). Su ventaja es que la URI documenta la propiedad y permite aplicar la regla de autorización "el propio usuario o un administrador"

Tabla III
MAPEO DE ENTIDADES DEL MODELO DE DATOS A RECURSOS REST

Entidad	Recurso (URI)	Decisión de diseño
Usuario	`/usuarios`, `/usuarios/:id`	Recurso de primer nivel. El registro y el inicio de sesión se separan en `/auth/*` porque no son operaciones CRUD sobre un usuario ya existente, sino procesos de acceso.
Receta	`/recetas`, `/recetas/:id`	Recurso de primer nivel con búsqueda y filtros como parámetros de consulta (`q`, `cocina`, `dificultad`, `pagina`), porque filtrar no crea un recurso nuevo. La respuesta incluye el promedio y el total de reseñas.
Reseña	`/recetas/:id/resenas` (listar, crear) y `/resenas/:id` (editar, borrar)	Anidada para crear y listar, porque una reseña no tiene sentido sin su receta y el padre queda explícito en la URI. Para editar y borrar se usa una URI plana, porque `id_resena` ya identifica unívocamente el recurso y repetir la receta añadiría redundancia y un caso de error (receta que no corresponde).
Colección	`/usuarios/:id/coleccion` y `/usuarios/:id/coleccion/:idReceta`	Anidada bajo el usuario por propiedad. Es un recurso asociativo: no tiene identificador propio, se identifica por el par (usuario, receta); por eso se agrega con `POST` (cuerpo `id_receta`) y se quita con `DELETE` sobre la receta.
Preparación	`/usuarios/:id/preparaciones`	Anidada bajo el usuario. Solo admite alta y consulta: una preparación es un hecho histórico y no se edita.
Hábitos	`/usuarios/:id/habitos`	*Endpoint compuesto* de solo lectura. No corresponde a ninguna tabla: agrega preparaciones por mes, por cocina, racha y actividad reciente.
Suscripción y plan	`/usuarios/:id/suscripcion`, `/planes`, `/suscripciones`	La suscripción vigente se trata como un recurso singular del usuario. El cambio a Premium se modela como `PUT` que fija el plan (`id_plan`) y no como una acción `POST/premium`, porque lo que se modifica es el estado del recurso. `/suscripciones` es la vista global del administrador y `/planes` es un catálogo de lectura.
Caja semanal	`/usuarios/:id/caja-semanal`	Anidada bajo el usuario y filtrable por `semana`. Incluye en la respuesta las recetas de la caja (relación resuelta por *incrustación*, no por referencias que obliguen a otra consulta).
Tipo de cocina, etiqueta	`/tipos-cocina`, `/etiquetas`	Catálogos de lectura pública y escritura restringida al administrador.
Panel	`/admin/resumen`	Endpoint compuesto de solo lectura (RF-15) que reúne conteos de usuarios, recetas, preparaciones y suscripciones.

una sola vez sobre `/usuarios/:id/*`. Su límite es la profundidad: se evitó pasar de dos niveles, y por eso la edición de reseñas es plana.

Las **referencias por identificador** se usan cuando la relación es entre recursos independientes. Una receta referencia su tipo de cocina con `id_tipo_cocina`, y para guardar una receta o marcarla como preparada el cuerpo lleva `id_receta`. Referenciar en el cuerpo, y no en la URI, evita URIs como `/usuarios/3/coleccion/7` para una operación de creación, donde el cliente aún no conoce un identificador de recurso propio. La API valida que la referencia exista y responde 404 o 400 según el caso, en lugar de dejar que la restricción de la base de datos produzca un error opaco.

Los **endpoints compuestos** se usan cuando el cliente necesita información que atraviesa varias tablas o que se calcula: los hábitos, el panel del administrador y el detalle de receta con calificación promedio. Reducen el número de solicitudes del cliente y mantienen la lógica de agregación en el servidor.

XIV-D. Endpoints y trazabilidad con los requerimientos

La API expone 37 operaciones sobre 22 rutas. La Tabla IV las agrupa y las vincula con los requerimientos funcionales del corte 1; con ello se comprueba que cada RF tiene al menos un endpoint que lo implementa. Las operaciones que no figuran con un RF propio (catálogos auxiliares) existen para sostener los datos que otros RF requieren, como los tipos de cocina que se asignan a una receta.

XIV-E. Autenticación, autorización y reglas de negocio

El inicio de sesión devuelve un token JWT firmado [6] que contiene el identificador y el rol del usuario, y que el cliente envía en el encabezado `Authorization: Bearer`. Se eligió un token sin estado en el servidor para no introducir una tabla de sesiones en este corte. Tres middlewares aplican las reglas de seguridad del corte 1: `autenticar` (RNF-05) rechaza con 401 las solicitudes sin token válido; `soloAdmin` (RNF-04) responde 403 a cuentas sin rol de administrador; y `propioOAdmin` restringe los recursos `/usuarios/:id/*` a su dueño o a un administrador. Así, un usuario que consulta la colección o los hábitos de otro recibe 403 y no los datos.

Las reglas de negocio explícitas del caso se implementan en la capa de servicios y se prueban en la colección: el plan gratuito permite hasta cinco recetas preparadas por mes (el valor es una decisión del equipo documentada en la Sección XII y se almacena en `planes`, no en el código); la caja semanal solo puede asignarse y consultarse para usuarios Premium y su semana debe iniciar en lunes; un usuario solo puede reseñar una vez cada receta; y solo el autor o un administrador puede modificar una reseña.

XIV-F. Validación y formato de errores

La validación de entrada se hace en un middleware previo al controlador, con reglas declarativas por recurso: campos obligatorios, tipo (cadena, entero, booleano, arreglo de enteros), longitudes, rangos (calificación de 1 a 5), enumera-

Tabla IV
ENDPOINTS DE LA API Y REQUERIMIENTOS FUNCIONALES QUE IMPLEMENTAN

Método	Ruta	Descripción	RF / acceso
POST	`/auth/registro`	Registra un usuario (plan gratuito)	RF-01, público
POST	`/auth/login`	Verifica credenciales y devuelve JWT	RF-02, público
GET	`/recetas`	Catálogo con búsqueda, filtros y paginación	RF-03, RF-04
GET	`/recetas/:id`	Detalle de receta	RF-05
POST, PUT, DEL.	`/recetas`, `/recetas/:id`	Alta, edición y baja de recetas	RF-16, RF-17, admin
POST	`/usuarios/:id/coleccion`	Guarda una receta en la colección	RF-06
GET, DEL.	`/usuarios/:id/coleccion`, `.../:idReceta`	Consulta y quita recetas guardadas	RF-06
POST, GET	`/usuarios/:id/preparaciones`	Marca receta preparada; lista preparaciones	RF-07
POST, GET	`/recetas/:id/resenas`	Crea y consulta reseñas de una receta	RF-08, RF-09
PUT, DEL.	`/resenas/:id`	Edita o elimina una reseña propia	RF-08
GET	`/usuarios/:id/habitos`	Indicadores de hábitos de cocina	RF-10
GET	`/usuarios/:id/suscripcion`	Plan vigente y uso del plan gratuito	RF-11
PUT	`/usuarios/:id/suscripcion`	Cambio de plan (a Premium)	RF-12
GET, POST, DEL.	`/usuarios/:id/caja-semanal` (y `/:idCaja`)	Consulta, asigna y elimina la caja semanal	RF-13
GET, PUT	`/usuarios/:id`	Consulta y edición de perfil	RF-14
GET	`/admin/resumen`	Panel de resumen	RF-15, admin
GET	`/suscripciones`	Suscripciones con búsqueda y filtros	RF-18, admin
GET, DEL.	`/usuarios`, `/usuarios/:id`	Lista, busca y elimina usuarios; PUT cambia rol y estado	RF-19, admin
CRUD	`/tipos-cocina`, `/etiquetas`	Catálogos auxiliares (lectura pública)	soporte, admin
GET	`/planes`	Planes disponibles	soporte, público

ciones (dificultad, rol, estado), formato de correo y de fecha. Las actualizaciones (`PUT`) aceptan cualquier subconjunto de campos, pero exigen al menos uno. Todos los errores comparten una estructura `{ error, detalles }`, donde `detalles` lista el campo y el mensaje de cada violación, de modo que el cliente pueda mostrar el problema junto al campo correspondiente (RNF-08).

XV. ARQUITECTURA DE LA API

XV-A. Organización por capas

El proyecto se organiza en capas con una dependencia estrictamente descendente: cada capa conoce solo a la inmediatamente inferior (Fig. 2). La separación persigue que un cambio en una preocupación (por ejemplo, la regla del límite mensual) afecte a un único archivo, y que cada capa pueda probarse de forma aislada.

La Tabla V detalla la responsabilidad de cada capa. La regla que ordena el diseño es que las capas superiores traducen HTTP y las inferiores no saben que HTTP existe: los repositorios devuelven filas, los servicios lanzan errores de dominio (`HttpError`) y solo el manejador central de errores los convierte en respuestas.

Tabla V
RESPONSABILIDADES POR CAPA

Capa	Responsabilidad
`routes/`	Asocia ruta y verbo con la cadena de middlewares y el controlador; no contiene lógica.
`middlewares/`, `validators/`	Autenticación y autorización por rol; validación del cuerpo y de los parámetros de ruta; manejo central de errores.
`controllers/`	Extraen datos de la solicitud, invocan al servicio y fijan el código de estado (201, 204, 200).
`services/`	Reglas de negocio (límites del plan, Premium, una reseña por receta), orquestación de varios repositorios y transacciones.
`repositories/`	Única capa que escribe SQL; consultas parametrizadas, sin lógica de negocio.
`config/`	Pool de conexiones a MySQL.

XV-B. Conexión a MySQL

El acceso a datos usa el controlador `mysql2` en su variante basada en promesas. En lugar de abrir una conexión por solicitud, se crea un *pool* con un máximo de diez conexiones reutilizables al iniciar la aplicación; las credenciales se leen de variables de entorno (`.env`, excluido del repositorio). Todas las consultas utilizan marcadores de posición (`?`), de modo que los valores viajan separados del texto SQL, lo que previene la inyección SQL. Las fechas se devuelven como texto (`dateStrings`) para evitar desplazamientos por zona horaria.

Las operaciones que modifican varias tablas se ejecutan en una *transacción* gestionada por una utilidad que obtiene una conexión del pool, confirma si todo termina bien y revierte ante cualquier error. Se usa en el registro (usuario y suscripción gratuita), en el cambio de plan (cancelar la suscripción vigente y crear la nueva), en la creación de reseñas con sus etiquetas y en la asignación de cajas con sus recetas; en todos los casos, un fallo a mitad de camino no deja datos a medias.

Los errores de la base de datos se traducen a códigos HTTP en el manejador central: una violación de unicidad produce 409, una referencia inexistente 400 y el borrado de un registro

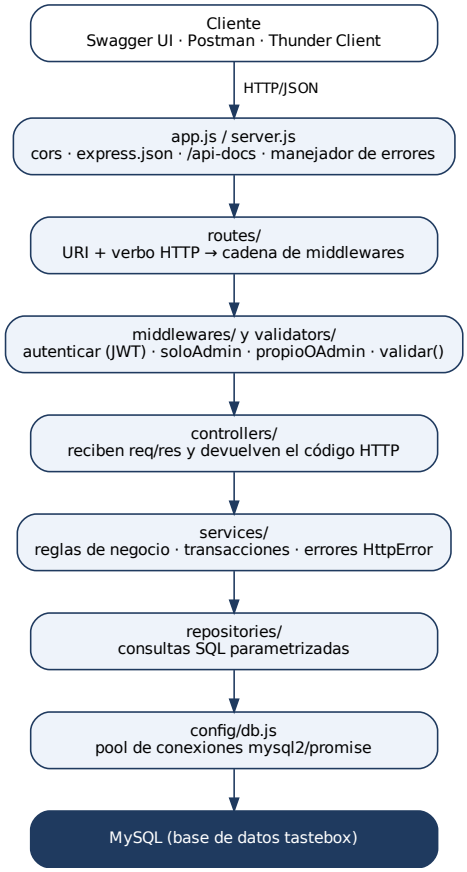


Figura 2. Arquitectura por capas de la API.

en uso 409. Los detalles internos de un error inesperado se registran en el servidor y no se envían al cliente (500 genérico).

XV-C. Estructura del proyecto

```
tastebox-api/
  database/schema.sql   esquema y datos
  src/
    server.js, app.js   arranque
    config/db.js        pool MySQL
    routes/             rutas por recurso
    middlewares/        auth, errores
    validators/         validacion de entrada
    controllers/        uno por recurso
    services/           reglas de negocio
    repositories/       consultas SQL
    docs/openapi.js     especificacion OpenAPI
    utils/              errores, transacciones
  scripts/              init-db, openapi, postman
  postman/              coleccion de pruebas
```

XV-D. Documentación y pruebas

La especificación OpenAPI 3.0 se mantiene en un único módulo que describe cada operación (parámetros, cuerpo, respuestas de éxito y de error con ejemplos [5]) y se publica con Swagger UI en la ruta `/api-docs`, donde cada endpoint puede ejecutarse de forma interactiva tras autorizar el token. La colección de pruebas contiene 99 solicitudes, con al menos un caso de éxito y uno de error por endpoint, y 248 aserciones sobre el código de estado y la forma de la respuesta. Se ejecuta de forma encadenada: el inicio de sesión guarda el token en variables de la colección y las altas guardan los identificadores que usan las solicitudes siguientes. En la última ejecución completa, las 248 aserciones pasaron y la colección puede repetirse sin reiniciar la base de datos, porque la limpieza final elimina los datos que ella misma crea.

REFERENCIAS

[1] OWASP Foundation, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[2] National Institute of Standards and Technology, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B. [Online]. Available: https://pages.nist.gov/800-63-3/sp800-63b.html

[3] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[4] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," IETF, RFC 9110, Jun. 2022.

[5] OpenAPI Initiative, "OpenAPI Specification v3.0.3," 2020. [Online]. Available: https://spec.openapis.org/oas/v3.0.3

[6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.